

Text Encryption Using Cryptographic Algorithms

Sections 3–4: High Level Design, Tools, and Development Phases

SecureText Proposed Solution

Academic Year 2025/2026

Abstract

This document completes the design part of the project by proposing **SecureText**, a web application for encrypting and decrypting textual information using modern cryptographic algorithms. The solution supports three modes: Fernet symmetric encryption, RSA asymmetric encryption, and hybrid encryption. The high level design describes the system goal, use cases, sequence flow, components, exchanged data, interactive explanation panel, tools, and development phases.

Contents

3	High Level Design Document for the Proposed Solution	3
3.1	Proposed Solution Overview	3
3.2	Problem Addressed by the Solution	3
3.3	System Goals	4
3.4	Actors and Use Cases	4
3.5	High Level Architecture	5
3.6	Main Components	6
3.6.1	Web User Interface	6
3.6.2	Application Controller	7
3.6.3	Fernet Service	7
3.6.4	RSA Service	7
3.6.5	Hybrid Encryption Service	8
3.6.6	Key Management Module	8
3.7	Interactive Application Experience	9

3.8	Functional Flow	10
3.8.1	Encryption Flow	10
3.8.2	Decryption Flow	11
3.9	Sequence Diagram	11
3.10	Detailed Mode Designs	12
3.10.1	Mode 1: Fernet Symmetric Encryption	12
3.10.2	Mode 2: RSA Asymmetric Encryption	13
3.10.3	Mode 3: Hybrid Encryption	14
3.11	API and Interface Design	14
3.11.1	General Encryption Request	15
3.11.2	General Encryption Response	15
3.11.3	General Decryption Request	15
3.11.4	General Error Response	16
	Deployment View	16
4	Tools and Development Phases	17
4.1	Selected Tools	17
4.2	Python Dependencies	17
4.3	Development Phases	18
4.3.1	Phase 1: Requirements and Design	18
4.3.2	Phase 2: Project Setup	18
4.3.3	Phase 3: Fernet Implementation	18
4.3.4	Phase 4: RSA Implementation	18
4.3.5	Phase 5: Hybrid Encryption	18
4.3.6	Phase 6: User Interface Integration	19
4.3.7	Phase 7: Testing and Validation	19
4.3.8	Phase 8: Final Delivery	19
4.4	Testing and Acceptance Criteria	19
	Limitations and Future Improvements	20
	Conclusion	21

3 High Level Design Document for the Proposed Solution

3.1 Proposed Solution Overview

The proposed solution is named **SecureText**. SecureText is a local web application that allows a user to enter text, choose an encryption method, generate or provide the required keys, encrypt the text, and decrypt it later using the correct key material.

The application is designed for an academic cybersecurity project. Its purpose is both practical and educational:

- **Practical purpose:** protect textual information by transforming readable plaintext into ciphertext that cannot be understood without the correct key.
- **Educational purpose:** demonstrate the difference between symmetric encryption, asymmetric encryption, and hybrid encryption.
- **Security demonstration:** show that strong encryption produces different ciphertexts even when the same plaintext is encrypted multiple times, because secure schemes use random values such as IVs, nonces, or padding.

The solution will support three cryptographic modes:

1. **Fernet symmetric encryption:** the same secret key is used for encryption and decryption.
2. **RSA asymmetric encryption:** a public key is used for encryption and the matching private key is used for decryption.
3. **Hybrid encryption:** the message is encrypted with Fernet, while the temporary Fernet session key is encrypted with RSA.

Security note. SecureText is not intended to replace production-grade password managers, secure messaging platforms, or enterprise key management systems. It is a controlled academic implementation that demonstrates correct cryptographic concepts using trusted libraries instead of manually implementing cryptographic primitives.

3.2 Problem Addressed by the Solution

Users often need to protect short pieces of sensitive information such as private notes, account recovery codes, confidential messages, or academic examples before sharing or storing them. Plaintext can be read immediately by any person or system that obtains access to it. SecureText addresses this problem by providing a controlled interface where text is converted into ciphertext using secure keys and standard cryptographic algorithms.

The main problem can be stated as follows:

How can a user securely encrypt and decrypt textual information while understanding the role of keys, algorithms, randomization, and integrity verification?

SecureText answers this problem through a web-based tool that:

- encrypts plaintext into ciphertext;
- decrypts ciphertext back into plaintext only when the correct key is provided;
- generates Fernet keys and RSA key pairs;
- demonstrates that identical plaintext does not produce identical ciphertext;
- explains the exchanged data for each encryption mode;
- handles invalid keys, corrupted ciphertext, and wrong decryption attempts safely.

3.3 System Goals

The main goals of SecureText are:

- Provide a simple web interface for text encryption and decryption.
- Implement Fernet encryption using the Python `cryptography` library.
- Implement RSA-2048 encryption using OAEP padding.
- Implement hybrid encryption by combining Fernet and RSA.
- Generate cryptographically secure keys using trusted library functions.
- Display outputs clearly so the user can understand plaintext, keys, ciphertext, and decrypted results.
- Demonstrate randomization by producing different ciphertexts for repeated encryption of the same plaintext.
- Validate inputs and return understandable error messages.

3.4 Actors and Use Cases

Table 1: Main actors and use cases

Actor	Use cases
User	Chooses an algorithm, generates or enters keys, encrypts text, decrypts text, views interactive explanations, checks randomized ciphertext, and receives validation feedback.

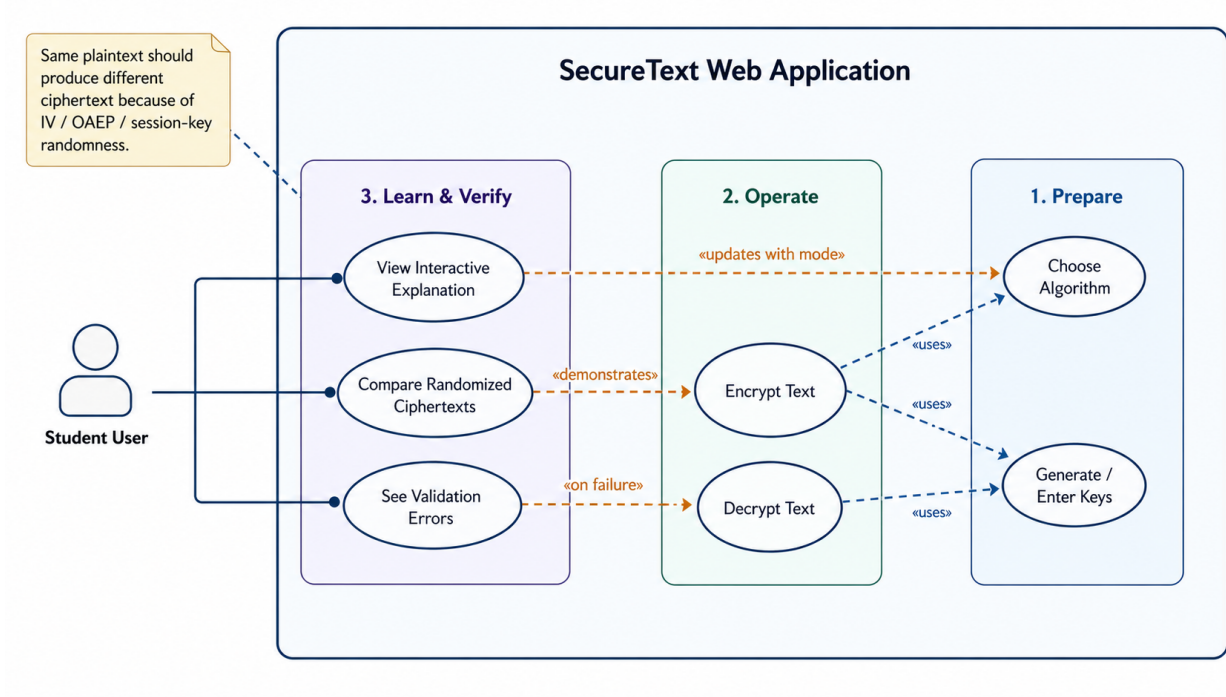


Figure 1: Use case diagram for SecureText

3.5 High Level Architecture

SecureText follows a simple layered architecture. The user interacts with the frontend through a browser. The frontend sends structured requests to the backend controller. The controller validates the request and calls the appropriate cryptographic service. Key generation and key formatting are handled by a key management module. The result is returned to the frontend and displayed to the user.

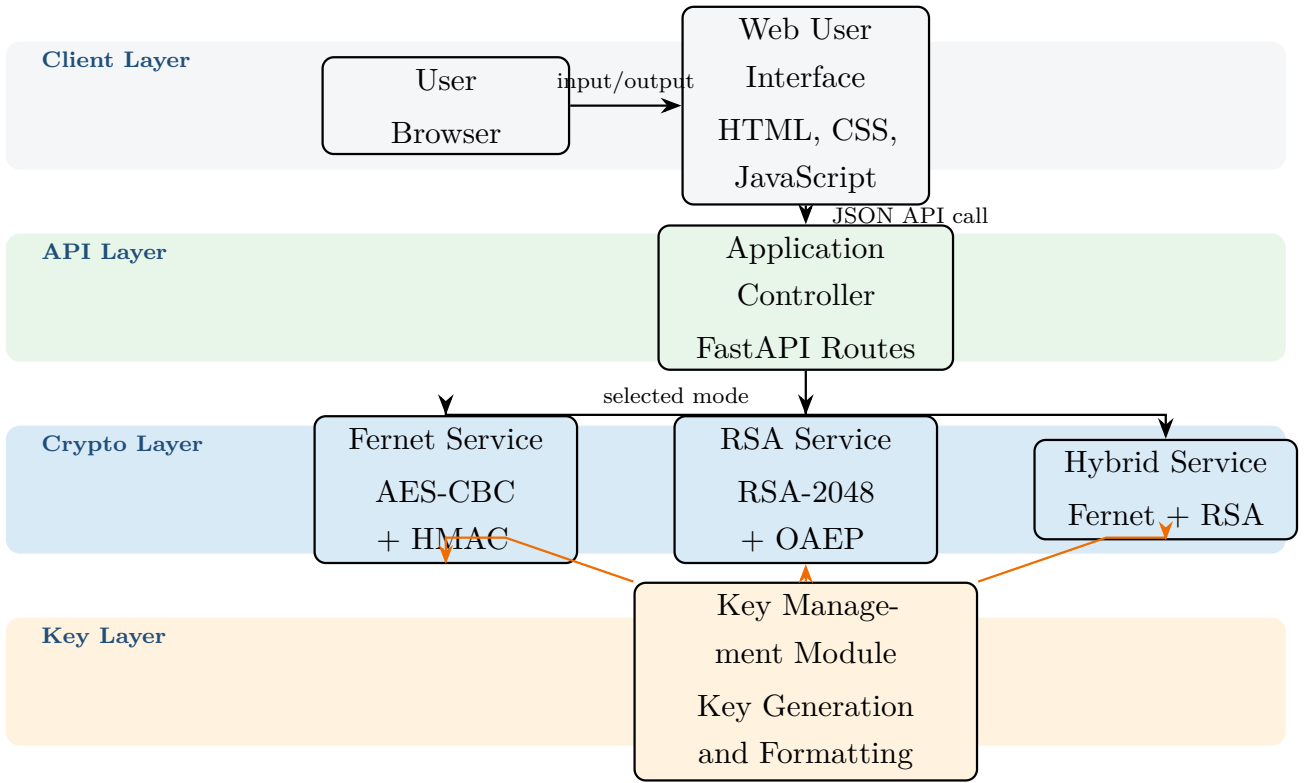


Figure 2: High-level architecture of SecureText

3.6 Main Components

3.6.1 Web User Interface

The web user interface is the part of the system visible to the user. It is implemented using HTML, CSS, and JavaScript. It contains:

- a plaintext input area;
- a ciphertext input/output area;
- an algorithm selector for Fernet, RSA, and Hybrid modes;
- buttons for key generation, encryption, decryption, and clearing fields;
- text areas for Fernet keys, RSA public keys, RSA private keys, encrypted session keys, and decrypted output;
- an interactive explanation panel showing the selected algorithm, required keys, encryption steps, decryption steps, and security warnings;
- visual feedback for successful operations and error messages.

The frontend does not implement the cryptographic algorithms itself. It sends user input to the backend and displays the returned result. Its interactive role is to guide the user through each mode: what key is needed, what data is sent, why the output changes, and what caused a failed decryption.

3.6.2 Application Controller

The application controller is responsible for receiving HTTP requests from the frontend. In a FastAPI implementation, it is represented by typed route functions. Its responsibilities include:

- reading JSON request bodies;
- validating required fields;
- selecting the correct cryptographic service;
- catching decryption and validation errors;
- returning JSON responses to the frontend.

The controller does not perform low-level cryptographic operations directly. Instead, it delegates these operations to dedicated services.

3.6.3 Fernet Service

The Fernet service performs symmetric encryption and decryption. It uses the **Fernet** recipe from the Python `cryptography` library. Fernet internally uses AES-128 in CBC mode for encryption and HMAC-SHA256 for integrity protection.

The Fernet service provides the following functions:

- generate a new Fernet key;
- encrypt plaintext using a Fernet key;
- decrypt a Fernet token using the same Fernet key;
- reject invalid tokens or modified ciphertext.

Security note. Fernet automatically includes a random IV in each token. Therefore, encrypting the same plaintext twice with the same key produces two different ciphertext tokens. This property directly satisfies the project requirement that strong encryption should produce different outputs even for the same input.

3.6.4 RSA Service

The RSA service performs asymmetric key generation, encryption, and decryption. It uses RSA-2048 with OAEP padding. OAEP is important because raw RSA encryption is not secure for real applications.

The RSA service provides:

- RSA-2048 key pair generation;
- public key export in PEM format;
- private key export in PEM format;
- encryption of short plaintext messages using the public key;
- decryption of RSA ciphertext using the private key.

RSA is slower than symmetric encryption and has a maximum message size. For RSA-2048 with OAEP-SHA256, the plaintext must be short. This limitation motivates the hybrid encryption mode.

3.6.5 Hybrid Encryption Service

The hybrid encryption service combines symmetric and asymmetric encryption. This is the approach used in many real systems because it benefits from the strengths of both algorithm families:

- Fernet encrypts the actual message efficiently.
- RSA encrypts only the temporary Fernet session key.

During hybrid encryption, the backend generates a fresh Fernet key for the message. It encrypts the plaintext with that Fernet key. Then it encrypts the Fernet key with the receiver's RSA public key. The final output contains two values:

1. the encrypted message;
2. the encrypted session key.

During hybrid decryption, the receiver uses the RSA private key to recover the Fernet session key, then uses the Fernet key to decrypt the message.

3.6.6 Key Management Module

The key management module handles temporary key generation and formatting. For the educational prototype, keys are displayed to the user so the user can understand the encryption and decryption process.

The module is responsible for:

- generating Fernet keys using a cryptographically secure random generator;
- generating RSA key pairs;

- serializing RSA public and private keys to PEM format;
- loading user-provided PEM keys;
- ensuring that private keys are used only for decryption;
- ensuring that public keys are used only for encryption.

Security note. In a production system, private keys should not be displayed openly in the web interface or stored in plaintext. For this academic prototype, key display is accepted because the goal is to demonstrate the cryptographic flow during a local demo.

3.7 Interactive Application Experience

SecureText includes an interactive explanation section beside the encryption form. When the user selects Fernet, RSA, or Hybrid mode, the page updates the visible fields and displays a short guided process for that algorithm. This makes the application more than a black-box encryption tool: it explains what is happening before and after the user clicks *Encrypt* or *Decrypt*.

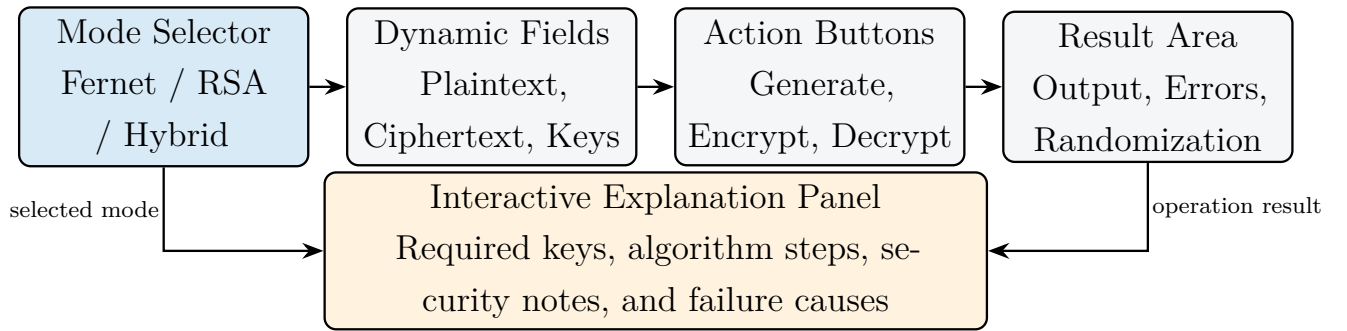


Figure 3: Interactive learning section inside the SecureText interface

Table 2: Interactive guidance shown for each algorithm

Mode	What the interface asks for	What the explanation panel highlights
Fernet	Plaintext or token, plus one shared Fernet key.	Same key encrypts and decrypts; random IV changes the token; HMAC detects modification.
RSA	Short plaintext or RSA ciphertext, plus public/private PEM key.	Public key encrypts; private key decrypts; OAEP adds randomness and protection.
Hybrid	Message, receiver public key, encrypted session key, and receiver private key.	Fernet encrypts the text; RSA protects the temporary Fernet key; this is the practical mode for longer text.

3.8 Functional Flow

The global functional flow is divided into two main operations: encryption and decryption.

3.8.1 Encryption Flow

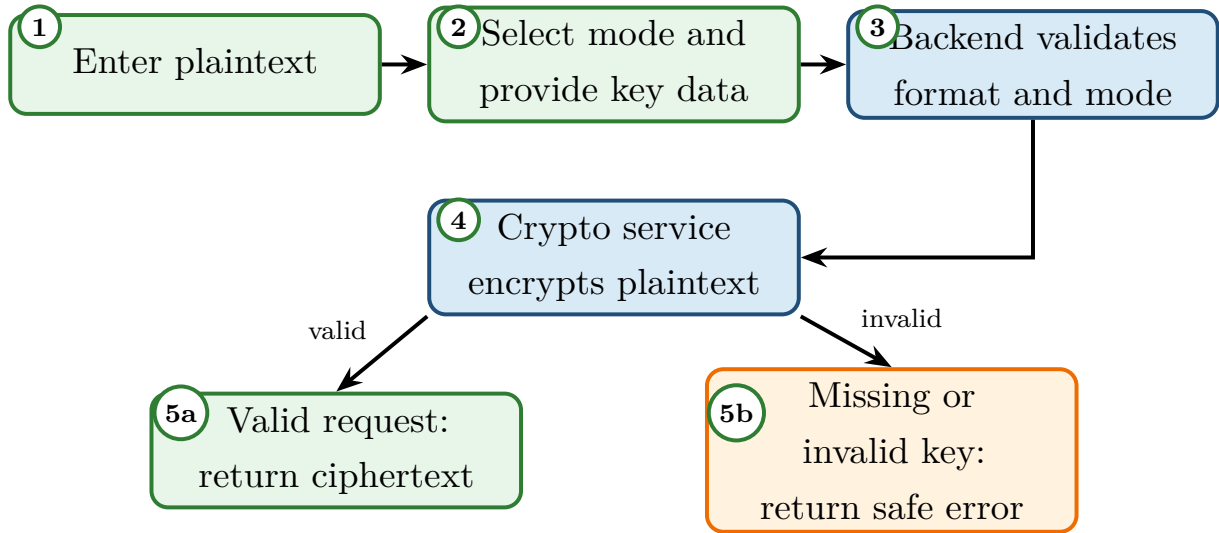


Figure 4: Encryption functional flow

The encryption process is:

1. The user enters the plaintext.
2. The user selects Fernet, RSA, or Hybrid mode.
3. The user generates or provides the required key.
4. The frontend sends the plaintext, selected mode, and key data to the backend.
5. The backend validates the request.
6. The backend calls the correct cryptographic service.
7. The backend returns the ciphertext and any generated values.
8. The frontend displays the result.

3.8.2 Decryption Flow

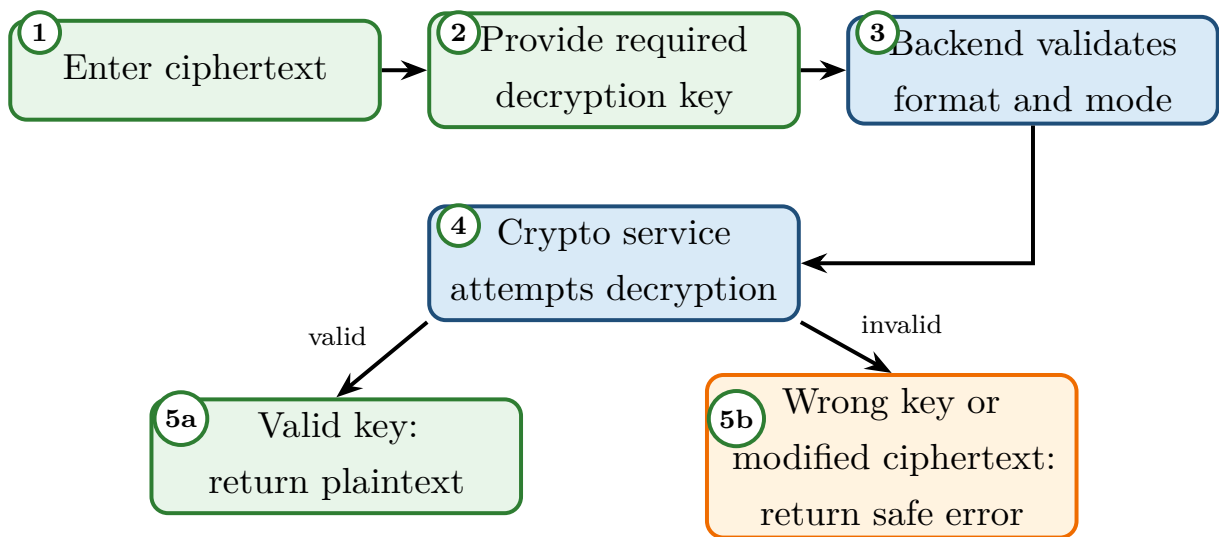


Figure 5: Decryption functional flow

The decryption process is:

1. The user enters or pastes ciphertext.
2. The user provides the correct key material.
3. The frontend sends the ciphertext, selected mode, and key data to the backend.
4. The backend validates the input format.
5. The selected service attempts decryption.
6. If the key and ciphertext are valid, plaintext is returned.
7. If the key is wrong or the ciphertext was modified, a safe error message is returned.

3.9 Sequence Diagram

The sequence diagram shows how one encryption or decryption request travels through the system. The same route structure is used for all three modes; only the selected cryptographic service changes.

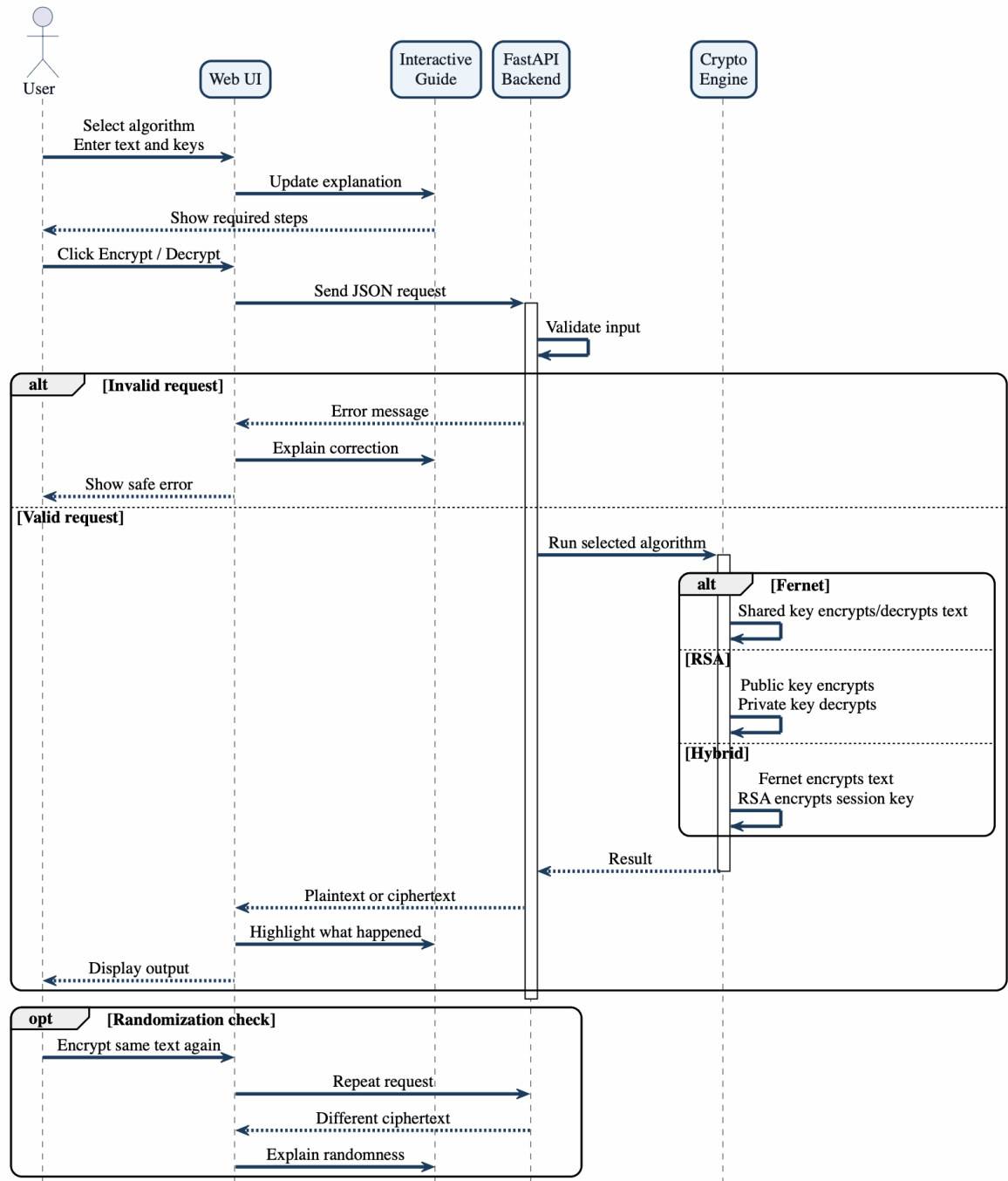


Figure 6: Sequence diagram for a SecureText encryption/decryption request

3.10 Detailed Mode Designs

3.10.1 Mode 1: Fernet Symmetric Encryption

Fernet mode uses one shared secret key. The same key is required for both encryption and decryption.

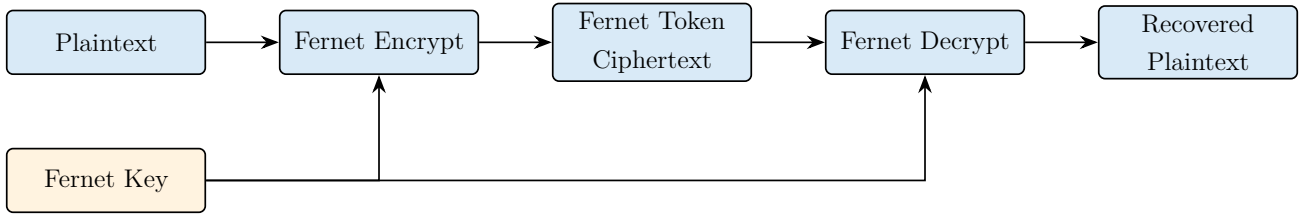


Figure 7: Intuitive Fernet flow: one shared key locks and unlocks the text

In the interface, the user sees Fernet as a shared-key lock: the same secret key must be present on both sides. The explanation panel highlights that Fernet adds a random IV and an integrity tag, so repeated encryption changes the token and tampering is detected.

Fernet exchanged data

Table 3: Fernet mode exchanged data

Operation	Data exchanged
Key generation request	No input required.
Key generation response	Fernet key encoded as URL-safe Base64.
Encryption request	Plaintext and Fernet key.
Encryption response	Fernet token/ciphertext.
Decryption request	Fernet token and the same Fernet key.
Decryption response	Recovered plaintext or invalid-token error.

3.10.2 Mode 2: RSA Asymmetric Encryption

RSA mode uses a key pair. The public key encrypts the message, and the private key decrypts it.

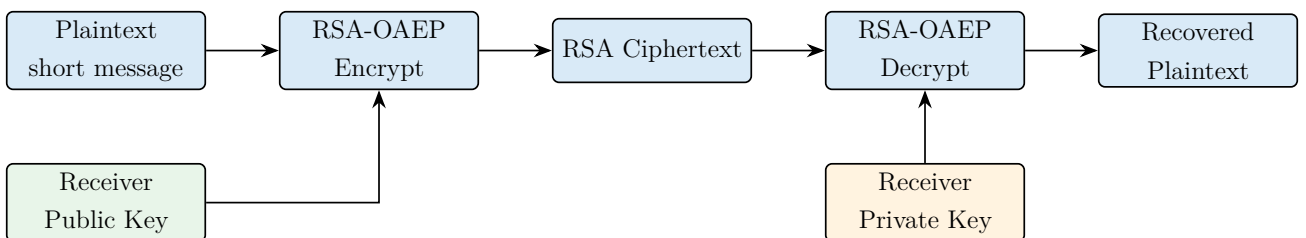


Figure 8: Intuitive RSA flow: the public key locks the message and the private key unlocks it

In the interface, RSA is presented as a public lock and a private unlock. The public key can be shared for encryption, while the private key stays secret for decryption.

RSA exchanged data

Table 4: RSA mode exchanged data

Operation	Data exchanged
Key pair generation request	No input required.
Key pair generation response	RSA public key and private key in PEM format.
Encryption request	Plaintext and receiver public key.
Encryption response	Base64-encoded RSA ciphertext.
Decryption request	RSA ciphertext and receiver private key.
Decryption response	Recovered plaintext or decryption error.

3.10.3 Mode 3: Hybrid Encryption

Hybrid encryption combines Fernet and RSA. It is the recommended mode for practical secure text exchange because it avoids the message-size limitation of RSA and uses efficient symmetric encryption for the actual text.

In the interface, Hybrid mode is explained as a package with two parts: the encrypted text and the encrypted session key. During decryption, RSA recovers the temporary Fernet key, then Fernet recovers the plaintext.

Hybrid exchanged data

Table 5: Hybrid mode exchanged data

Operation	Data exchanged
Encryption request	Plaintext and receiver RSA public key.
Encryption response	Fernet encrypted message and RSA encrypted Fernet session key.
Decryption request	Encrypted message, encrypted session key, and receiver RSA private key.
Decryption response	Recovered plaintext or controlled decryption error.

3.11 API and Interface Design

The first implementation can use a simple FastAPI backend with JSON endpoints. The exact route names can be adapted, but the following interface is recommended because it keeps the frontend clear and easy to test.

Table 6: Recommended backend API endpoints

Endpoint	Method	Purpose
/	GET	Serves the main web page.
/api/fernet/key	POST	Generates a new Fernet key.
/api/rsa/keypair	POST	Generates a new RSA-2048 public/private key pair.
/api/encrypt	POST	Encrypts plaintext using the selected mode.
/api/decrypt	POST	Decrypts ciphertext using the selected mode.

3.11.1 General Encryption Request

Listing 1: Example encryption request

```
{
  "mode": "fernet | rsa | hybrid",
  "plaintext": "Text entered by the user",
  "fernet_key": "required only for Fernet mode",
  "public_key": "required for RSA and Hybrid modes"
}
```

3.11.2 General Encryption Response

Listing 2: Example encryption response

```
{
  "success": true,
  "mode": "hybrid",
  "ciphertext": "base64-or-fernet-token-value",
  "encrypted_session_key": "base64-value-for-hybrid-mode",
  "message": "Encryption completed successfully"
}
```

3.11.3 General Decryption Request

Listing 3: Example decryption request

```
{
  "mode": "fernet | rsa | hybrid",
  "ciphertext": "ciphertext submitted by the user",
  "fernet_key": "required only for Fernet mode",
  "private_key": "required for RSA and Hybrid modes",
}
```

```
{  
  "encrypted_session_key": "required only for Hybrid mode"  
}
```

3.11.4 General Error Response

Listing 4: Example error response

```
{  
  "success": false,  
  "error": "Invalid key, corrupted ciphertext, or unsupported mode"  
}
```

Deployment View

The executable deliverable is a local web application. The user runs the FastAPI server on a local machine and opens the application in a browser.

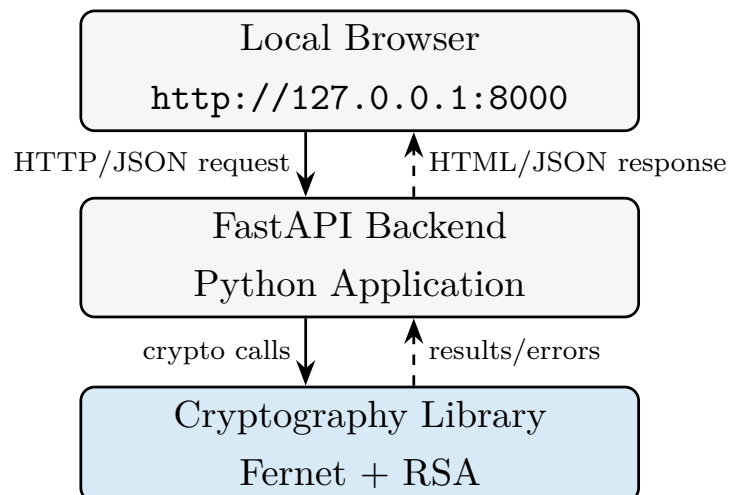


Figure 10: Local deployment view

4 Tools and Development Phases

4.1 Selected Tools

Table 7: Tools selected for implementing SecureText

Category	Tool	Justification
Programming language	Python	Simple syntax, strong library ecosystem, suitable for academic prototypes.
Web framework	FastAPI	Modern Python framework with clear JSON APIs, automatic validation, and interactive API documentation.
Cryptography library	cryptography	Provides high-level Fernet and secure RSA/OAEP primitives.
Frontend	HTML, CSS, JavaScript	Standard browser technologies for forms, buttons, and dynamic output.
Data format	JSON	Simple format for exchanging data between frontend and backend.
Testing	pytest	Useful for testing encryption, decryption, and failure scenarios.
Documentation	LaTeX / Overleaf	Professional formatting for the final academic report.
Version control	Git	Tracks implementation changes and supports team collaboration.

4.2 Python Dependencies

The recommended `requirements.txt` file is:

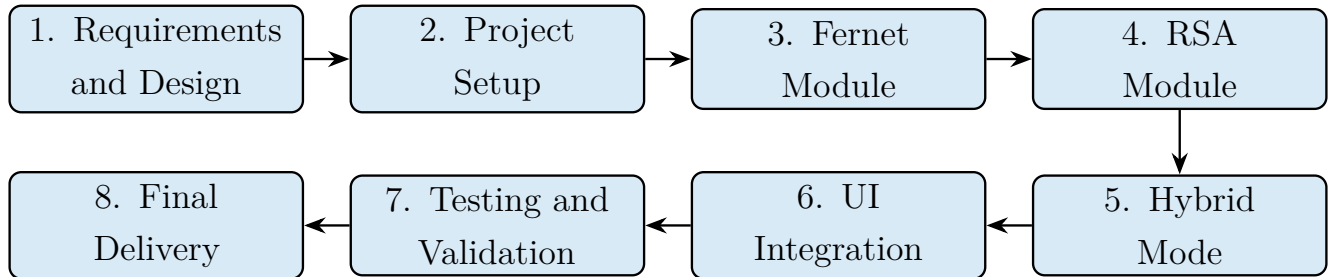
Listing 5: Recommended requirements.txt

```
fastapi==0.111.0
uvicorn==0.30.1
cryptography==42.0.8
pytest==8.2.2
```

The dependency versions can be updated if the development environment already uses newer stable versions, but the group should avoid using deprecated cryptographic APIs.

4.3 Development Phases

The project will be developed in eight compact phases. Each phase produces a visible output that can be checked before moving forward.



Development phases roadmap

4.3.1 Phase 1: Requirements and Design

Define the three modes, required inputs, outputs, API endpoints, UI states, and expected error cases. **Output:** approved HLD, diagrams, and exchanged-data tables.

4.3.2 Phase 2: Project Setup

Create the FastAPI project, install dependencies, serve the first HTML page, and prepare service modules. **Output:** local app running at `http://127.0.0.1:8000`.

4.3.3 Phase 3: Fernet Implementation

Implement Fernet key generation, encryption, decryption, invalid-token handling, and repeated-encryption comparison. **Output:** working Fernet endpoints and tests.

4.3.4 Phase 4: RSA Implementation

Implement RSA-2048 key generation, PEM import/export, RSA-OAEP encryption, decryption, and wrong-key handling. **Output:** working RSA endpoints and tests.

4.3.5 Phase 5: Hybrid Encryption

Generate a fresh Fernet session key per message, encrypt the message with Fernet, and encrypt the session key with RSA. **Output:** hybrid package creation, decryption, and tests.

4.3.6 Phase 6: User Interface Integration

Connect mode selection, key fields, action buttons, result areas, and the interactive explanation panel to the backend. **Output:** complete browser interface.

4.3.7 Phase 7: Testing and Validation

Validate correct encryption/decryption, wrong keys, modified ciphertext, malformed inputs, and randomized ciphertext output. **Output:** passing tests and manual checklist.

4.3.8 Phase 8: Final Delivery

Package the code, final PDF, screenshots, README, and demo script. **Output:** executable deliverable ready for evaluation.

4.4 Testing and Acceptance Criteria

Table 8: Acceptance criteria for SecureText

Feature	Test scenario	Expected result
Fernet encryption	Encrypt plaintext with a generated Fernet key.	A Fernet token is produced.
Fernet decryption	Decrypt the token with the same key.	Original plaintext is recovered.
Fernet wrong key	Decrypt the token with a different key.	Decryption fails with controlled error.
Randomization	Encrypt the same plaintext twice with the same mode.	The ciphertext values are different.
RSA key generation	Generate an RSA key pair.	Public and private PEM keys are returned.
RSA encryption	Encrypt a short message with the public key.	Base64 ciphertext is produced.
RSA decryption	Decrypt using the matching private key.	Original plaintext is recovered.
RSA wrong key	Decrypt with a non-matching private key.	Decryption fails safely.

Feature	Test scenario	Expected result
Hybrid encryption	Encrypt a message with recipient public key.	Encrypted message and encrypted session key are produced.
Hybrid decryption	Decrypt with encrypted session key and private key.	Original plaintext is recovered.
Malformed input	Submit invalid keys or empty fields.	Application returns clear error messages.

Limitations and Future Improvements

The first version of SecureText is intentionally simple. Its main limitations are:

- keys are displayed in the browser for demonstration purposes;
- no user login or access control is implemented;
- no database is used for storing encrypted messages;
- private keys are not protected by a password;
- the application is intended for local execution, not public deployment.

Future improvements could include:

- password-protected private keys;
- secure storage of encrypted messages;
- user authentication;
- audit logs for encryption and decryption actions;
- support for file encryption;
- use of AES-GCM as another authenticated encryption mode;
- optional cloud key management integration.

Conclusion

SecureText is a complete proposed solution for the text encryption project. It provides a clear web-based workflow for encrypting and decrypting text using Fernet, RSA, and hybrid cryptography.

The high level design separates the user interface, controller, cryptographic services, and key management module. The data exchange model explains exactly what is sent and returned in each mode. The chosen tools are appropriate for a beginner-friendly but secure implementation, and the development phases provide a practical path from design to executable deliverable.